TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

CÔNG NGHỆ MÃ NGUỒN MỞ

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Chương 3

LẬP TRÌNH SHELL

Mục tiêu

- Phân biệt các loại Shell trong Linux
- Ngôn ngữ lập trình shell
- Viết được chương trình shell cơ bản

Nội dung

1. Giới thiệu Shell
2. Cú pháp ngôn ngữ lập trình Shell
3. Tham số và Biến số trong Shell
4. Cấu trúc điều khiển
5. Các phép toán số học
6. Xử lý chuỗi, văn bản
7. Hàm
8. Mảng

1 Giới thiệu Shell

1.1 Khái niệm Shell

- Shell là một chương trình đặt biệt cung cấp giao diện để người dùng sử dụng HĐH hành giao tiếp với phần cứng máy tính.
- Shell nhận những lệnh từ người dùng và thông dịch thành lệnh mà kernel có thể hiểu được.
- Shell bắt đầu thực hiện khi người dùng đăng nhập máy tính hoặc khi thực hiện các lệnh tại terminal.
- Về cơ bản, Shell trong Linux giống với Command Prompt trong Windows.
- Trong Linux có nhiều loại shell và người dùng có thể chuyển đổi giữa các Shell

1 Giới thiệu Shell

1.1 Khái niệm Shell

Các loại Shell trong Linux:

- **Bourne Shell (sh):**
  - Do Steven Bourne viết.
  - Là Shell có mặt trên hầu hết các hệ thống Unix/Linux
  - Có phần mở rộng .sh
- **Bash shell (Bourne Again Shell):**
  - Có phần mở rộng .bash.
  - Kế thừa từ Bourn Shell
  - Nó có giao diện lập trình rất mạnh và linh hoạt.
  - Shell được cài đặt mặc định trên các hệ thống Linux.

1 Giới thiệu Shell

1.1 Khái niệm Shell

Các loại Shell trong Linux:

- **csh (C Shell):**
  - Thuộc trường đại học Berkeley. Thường đi kèm với HĐH BSD Unix.
  - tcsh là version cải tiến của csh.
  - Hỗ trợ rất mạnh cho những Programmer C
- **ksh (Korn Shell):**
  - Do David Korn.
  - Kết hợp tính năng ưu việt của sh và csh…
- Ngoài ra còn có một số Shell khác như: ssh, nfssh, mcsh…
- Để xem Shell đang chạy, sử dụng lệnh: echo $SHELL hoặc echo $0

1.2 Khái niệm Shell scripting

- Shell Scripting là một chương trình máy tính mã nguồn mở được viết để thực hiện các công việc với shell.
- Shell Scripting là chương trình viết một chuỗi lệnh để shell thực thi.
- Kết hợp các chuỗi lệnh dài và lặp đi lặp lại thành một tập lệnh duy nhất và đơn giản có thể được lưu trữ và thực thi bất cứ lúc nào.
- Mục đích của Shell Scripting:
  - Hạn chế việc thực thi code nhiều lần.
  - Dễ dàng backup hệ thống thường xuyên.
  - Giám sát hệ thống.
  - Bổ sung chức năng vào shell.

1.2 Khái niệm Shell Scripting

Ưu điểm của shell scripting

- Ngôn ngữ thông dịch: dễ dàng dò tìm và phát hiện lỗi; thay đổi chỉnh sửa script không cần phải biên dịch lại từ đầu.
- Ngôn ngữ phù hợp với ứng dụng: đường uống (pipe line), thay thế lệnh và các biến trong môi trường Unix
- Cho phép kết nối các thành phần với các ngôn ngữ khác: Input-Output các biến, lỗi, kết quả…

1.2 Khái niệm Shell Scripting

Nhược điểm của shell scripting

- Bắt nguồn từ Unix, ngôn ngữ lập trình dùng cú pháp của shell không dễ tiếp cận cho người mới bắt đầu.
- Thêm hoặc bớt khoảng trắng trong câu lệnh sẽ gây lỗi cú pháp.
- Bash sở hữu nhiều cú pháp để thao tác cùng một chức năng. Vì vậy, đôi khi gây ra sự không tương thích với các phiên bản shell.
- Các ký tự đặc biệt dùng trong câu lệnh sẽ có ý nghĩa khác nhau trong từng hoàn cảnh khác nhau, gây khó khăn khi sử dụng.

Open Source Technologies

1.3 Cách thực hiện Shell Scripting

Cách 1: Gõ trực tiếp dòng lệnh tại terminal

Ví dụ:

```
$for file in *
> do
> if grep -l 'main()' $file
> then
> more $file
> fi
> done
```

Có thể viết liên tục các lệnh phân cách bởi dấu ";"

```
$for file in *;do;if grep -l 'main()' $file; then; more $file;fi;done
```

1.3 Cách thực hiện Shell Scripting

Cách 2:

- Viết các câu lệnh vào một file văn bản với phần mở rộng .bash hoặc .sh
- Thực hiện file văn bản này nhưng một chương trình phần mềm.
- Cấp quyền thực thi cho file script
- Sử dụng một trong các cách sau để chạy file script:
  - sh Tên_file_script
  - bash Tên_file_script
  - ./Tên_file_script

1.3 Cách thực hiện Shell Scripting

Ví dụ thực hiện file shell script mẫu

```
#!/bin/bash

echo "Nhap vao ten cua ban:"
read Ten
echo "Xin chao $Ten"
```

1. Sử dụng **nano** để soạn thảo file như trên. Nhấn **Ctrl + o** để lưu với tên hello.sh
2. Cấp quyền excute cho file

   chmod +x hello.sh
3. ./hello.sh

Lưu ý: File script luôn bắt đầu bằng #! /bin/bash hoặc #! /bin/sh

Phải cấp quyền excute cho file bash.

2 Cú pháp ngôn ngữ Shell

Một số lệnh cơ bản của ngôn ngữ lập trình Shell

read => Nhập dữ liệu từ bàn phím

echo => Xuất dữ liệu ra màn hình

=> chú thích trong ngôn ngữ shell

“*” đại diện cho một chuỗi (có thể rỗng) các ký tự bất kỳ

“?” đại diện cho một ký tự bất kỳ

“[ ]” đại diện cho một ký tự bất kỳ trong ngoặc vuông, có thể sử dụng dấu - để liệt kê một khoảng ký tự liền nhau

2 Cú pháp ngôn ngữ Shell

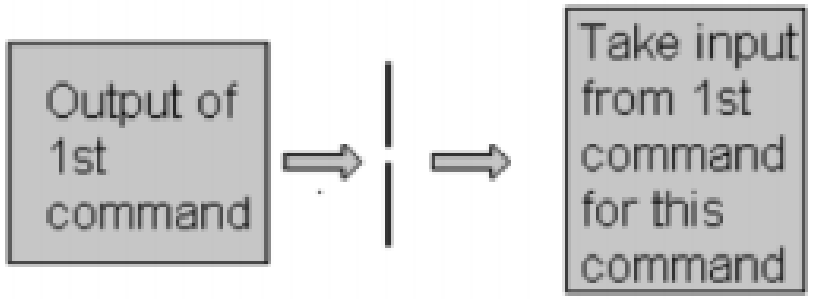
Chuyển hướng Input ⇔ Output

- Trong Linux có thể chuyển kết quả output của lệnh này thành input của lệnh khác
- Có 3 loại chuyển hướng: > ; >>; <
  - Symbol >: Lấy output cho lệnh tiếp theo
  - $ ls > myfiles # ghi đè thông tin nếu đã tồn tại file
- Symbol >>: Output kết hợp với lệnh tiếp theo
  - Syntax: Linux-command >> filename
  - $ date >> myfiles # thêm thông tin vào phần cuối của file
- Symbol <
  - Linux-command < filename
  - $ cat < myfiles # Nhận đầu vào lệnh từ tập tin

2 Cú pháp ngôn ngữ Shell

Đường ống (pipeline)

- Ống dẫn là cách để kết nối output của một chương trình thành input của một chương trình khác mà không dùng file tạm
- Ống dẫn thường dùng để thực thi nhiều hơn 1 câu lệnh trên cùng một dòng.
- Cú pháp : command1 | command2 | command3



Chi tiết lệnh xuất ra màn hình echo

Cú pháp: **echo** [options] [string, variables...]

- **-e** : cho phép hiểu những ký tự theo sau dấu trong chuỗi
- **\n** : xuống dòng
- **-n** : không in ký tự xuống dòng.
- **\a** : alert (tiếng chuông)
- **\b** : backspace
- **\c** : không xuống dòng
- **\r** : về đầu dòng
- **\t** : tab

Ví dụ:

```
echo -e “Cong \tnghe \tMMM“
echo -e “Cong \cnghe ma nguon mo"
```

2 Cú pháp ngôn ngữ Shell

Open Source Technologies

Dấu nháy đôi “” và dấu nháy đơn ‘’

- Dấu nháy kép - “Double Quotes”: Nội dung trong “” được xem là chuỗi (ngoại trừ $Biến hoặc ký tự đặc biệt \).
- Dấu nháy đơn – ‘Single quotes’: Nội dung trong ‘’ được xem là chuỗi, kể cả \ và $
- Dấu nháy huyền - `Back quotes` (ở phím ~): thực thi câu lệnh

Ví dụ:

- n=10
- echo **'**So vua nhap la: $n**'** => In ra câu So vua nhap la: $n
- echo **“**So vua nhap la: $n**”** => In ra câu So vua nhap là: 10
- echo "Today is `date`". => In ra Today is Thứ Hai, 10 Tháng 10 năm 2023

3 Tham số và Biến số

Open Source Technologies

Các loại tham biến trong ngôn ngữ lập trình shell

- Biến thông thường
- Biến môi trường
- Tham số

3 Tham số và Biến số

Open Source Technologies

3.1 Biến thông thường

- Biến không cần phải khởi tạo trước khi sử dụng.
- Mặc định, giá trị trong biến luôn luôn là kiểu chuỗi
- Tên biến phân biệt chữ hoa chữ thường.
- Xuất biến ra màn hình, dùng lệnh echo $Tên_biến
- Nhập giá trị biến từ bàn phím: dung lệnh read Tên_biến
- Lấy giá trị của biến: them dấu $ vào trước tên biến $test
- Gán giá trị cho biến.
  - Cú pháp: Tên_biến=giá_trị

Lưu ý không có khoảng trắng trước và sau dấu =

3 Tham số và Biến số

3.1 Biến thông thường

Quy tắc đặt tên biến trong SHELL

- Tên biến nên bắt đầu bằng chữ hoặc dấu gạch dưới (_)
- Không bắt đầu bằng số.
- Không chứa khoảng trắng hoặc các ký tự đặc biệt: \, $, &, *,?…
- Không trùng với những keyword hay biến môi trường. Ví dụ: if, else, HOME, PATH…
- Không trùng với những câu lệnh của shell. Ví dụ: dir, ls, cp, rm…

3 Tham số và Biến số

3.1 Biến thông thường

Ví dụ khai báo biến hợp lệ

- Name=CongNghe
- Count=10
- _var=Ma_Nguon_Mo
- myCar=Honda
- MY_VAR=BMW

Ví dụ khai báo biến không hợp lệ

- Name=Cong Nghe
- 10=count
- 2var= Ma_Nguon_Mo
- My Car = Honda
- MY\VAR=BMW

3.2 Biến môi trường

- Là biến được khai báo sẵn và gán giá trị mặc định khi shell được khởi động.
- Thường biến môi trường được viết hoa
- Xem danh sách biến môi trường:

  env hoặc **printenv**

- Tạo biến môi trường:

  export name=value hoặc

- Sử dụng biến môi trường: **echo $TÊN_BIẾN**

3 Tham số và Biến số

3.2 Biến môi trường

Một vài biến môi trường thường được sử dụng trong Linux

Environment	Description
$USER	Tên user
$PATH	Danh sách đường dẫn các thư mục tìm kiếm khi thực hiện lệnh
$HOME	Đường dẫn thư mục home
$PWD	Đường dẫn làm việc hiện hành
$LANG	Ngôn ngữ mặc định của hệ thống
$EDITOR	Chương trình chỉnh sửa file mặc định
$UID	Lấy ID của user hiện tại
$SHELL	Vị trí chạy các shell của user hiện tại.
$HOSTNAME	Tên máy tính

3.3 Tham số

- Tham số được gọi là các biến được xác định trong một khai báo hàm.
- Tham số được sử dụng để nhận các đối số được truyền vào khi gọi hàm.
- Những tham số được xây dựng sẵn.

$#	: tổng số tham số có trong file script
$*	: danh sách tên của tất cả các tham số.
$0	: tên tập tin lệnh. Sử dụng $0 để lấy tên file script
$1, $2, ..,$9	: giá trị các biến tham số thứ 1, thứ 2, …, thứ 9

Open Source Technologies

3.3 Tham số

Ví dụ về tham số trong shell

Tập tin user.sh có nội dung như sau:

```
#!/bin/bash

echo “User: $1";
echo “Tuoi: $2";
echo “Ten nguoi dung: $3";
```

Thực hiện chạy script không truyền tham số

./user.sh

Kết quả:

User:
Tuoi:
Ten nguoi dung:

Thực hiện chạy script có truyền tham số

./user.sh sinhvien 20 “Nguyen Thanh Binh”

Kết quả:

User: sinhvien
Tuoi: 20
Ten nguoi dung: Nguyen Thanh Binh

4 Cấu trúc điều khiển

4.1 Lệnh kiểm tra điều kiện

- Trong ngôn ngữ lập trình shell, để kiểm tra điều kiện đúng hay sai ta sử dụng lệnh test hoặc [ ]. Trả về 1 nếu FALSE, trả về 0 nếu TRUE.
- Shell cho kiểm tra trên 4 loại: Biến, Tập tin, Số, Chuỗi

Ví dụ về lệnh kiểm tra chuỗi

```
string1="HELLO Sir"
string2="Hello"
[ $string1 > $string2 ] => trả về True (0)
```

Ví dụ về lệnh kiểm tra số

```
var=10
[ $var -eq 0 ] => FALSE (1)
```

Hoặc `test $var –eq  0 => false (1)`

Để kiểm tra kết qua ta dùng lệnh: echo $?

Lưu ý: Sau dấu [và trước dấu] phải có khoảng trắng

4.1 Lệnh kiểm tra điều kiện

Kiểm tra Biến

Cú pháp: [[-z|-n $Tên_biến]]

Tham số	Ý nghĩa
-z	Kiểm tra biến có rỗng hay không? Rỗng trả về TRUE Không rỗng trả về FALSE
-n	Kiểm tra biến có không rỗng hay không? Không rỗng trả về TRUE Rỗng trả về FALSE

Ví dụ:

so=10

test –z $so => Trả về FALSE (1)

4.1 Lệnh kiểm tra điều kiện

Kiểm tra chuỗi

Cú pháp: [[string1 =|!=|<|> string2]]

Tham số	Ý nghĩa
=	Kiểm tra bằng
!=	Kiểm tra không bằng
>	Kiểm tra lớn hơn
<	Kiểm tra nhỏ hơn

4.1 Lệnh kiểm tra điều kiện

Kiểm tra tập tin

Tham số	Ý nghĩa
-d Tên_file	True nếu là thư mục
-e Tên_file	True nếu Tên_file tồn tại trên đĩa
-f Tên_file	True nếu Tên_file là tập tin thông thường
-g Tên_file	True nếu set-group-id() được thiết lập cho file
-r Tên_file	True nếu Tên_file cho phép đọc
-s Tên_file	True nếu kích thước Tên_file khác 0
-u Tên_file	True nếu set-user-id() được thiết lập cho file
-w Tên_file	True nếu Tên_file cho phép ghi
-x Tên_file	True nếu Tên_file cho phép thực thi

4 Cấu trúc điều khiển

4.1 Lệnh kiểm tra điều kiện

Kiểm tra số

Cú pháp: [["num1" -eq|-ne|-gt|-lt "num2"]]

Tham số	Ý nghĩa
-eq	So sánh bằng (equal)
-ne	So sánh không bằng (not equal)
-gt	So sánh lớn hơn (greater than)
-ge	So sánh lớn hơn hoặc bằng
-lt	So sánh nhỏ hơn (less than)
-le	So sánh nhỏ hơn hoặc bằng

4 Cấu trúc điều khiển

Open Source Technologies

4.1 Lệnh kiểm tra điều kiện

Một vài ví dụ về cách sử dụng

test -f name => Kiểm tra name có phải là tập tin hay không?

Hoặc có thể viết [-f $name]

test -d name => Kiểm tra name có phải là thư mục hay không?

test String1=String2 => So sánh String1 có bằng String2 hay không

test String1 != String2 => So sánh String1 có khác String2 hay không

4 Cấu trúc điều khiển

Open Source Technologies

4.1 Lệnh kiểm tra điều kiện

Ký tự kết hợp AND, OR

Tham số	Ý nghĩa
-a hoặc &&	AND – Đúng khi cả hai điều kiện cùng đúng
-o hoặc ||	OR – Sai khi cả hai điều kiện cùng sai

Ví dụ cách viết:

```
[[ -d "/etc" && -x "/etc" ]]
(TEST1 -a TEST2) -a TEST3
```

```
if [ $first -le 10 ] || [ $second -gt 20 ]
then
    echo "At least one conditions is true"
else
    echo "Both conditions are failed"
fi
```

Open Source Technologies

4.2 Cấu trúc rẽ nhánh

Cấu trúc if

```
if   <biểu thức điều kiện>
then
      <lệnh 1>
else
      <lệnh 2>
fi
```

Ví dụ: Nhập điểm cho môn học và xuất kết quả.
Tạo tập tin ketqua.sh có nội dung sau:

```
echo “Chuong trinh ket qua mon hoc”
echo “Nhap vao diem”
read diem
if  [ $diem  -ge  5 ]
then
      echo  “Dat”
else
      echo  “Khong dat”
fi
```

Open Source Technologies

4.2 Cấu trúc rẽ nhánh

Cấu trúc elif

```
if      <biểu thức điều kiện>
then
        <lệnh 1>
elif    <biểu thức điều kiện>
then
        <lệnh 1>
else
        <lệnh 1>
fi
```

```
echo "Nhap vao 2 so"
echo "So thu nhat"
read so1
echo "So thu hai"
read so2
if [ $so1 -gt $so2 ]
then
   echo "So thu 1 lon hon so thu 2"
elif [ $so1 -lt $so2 ]
then
   echo "So thu 1 nho hon so thu 2"
else
   echo "Hai so bang nhau"
fi
```

Open Source Technologies

4.2 Cấu trúc rẽ nhánh

Một vài ví dụ về cấu trúc rẽ nhánh

```
#!/bin/sh

echo "Is it morning? Please answer yes or no"
read timeofday

if [ $timeofday = "yes" ]; then
   echo "Good morning"
else
   echo "Good afternoon"
fi

exit 0
```

4.2 Cấu trúc rẽ nhánh

```
#!/bin/sh

echo "Is it morning? Please answer yes or no"
read timeofday

if [ $timeofday = "yes" ]; then
   echo "Good morning"
elif [ $timeofday = "no" ]; then
   echo "Good afternoon"
else
   echo "Sorry, $timeofday not recognized. Enter yes or no"
   exit 1
fi

exit 0
```

4.3 Cấu trúc lựa chọn case

```
case   <biến>  in
        < giá_trị_ 1>)
                <lệnh_1> ;;
        < giá_trị_ 2>)
                <lệnh_2> ;;
        <giá_trị_3>)
                <lệnh_3> ;;
esac
```

Ví dụ: Tạo menu cho phép người dùng chọn

Tạo tập tin thucdon.sh có nội dung sau:

```
echo  "THUC DON"
echo  "1. Liet ke thu muc hien hanh"
echo  "2. Duong dan thu muc hien hanh"
read  chon
case  $chon  in
      1) ls –l ;;
      2) pwd ;;
      *) echo "Khong hop le" ;;
esac
```

4.3 Cấu trúc lựa chọn case

```
#!/bin/sh

echo "Is it morning? Please answer yes or no"
read timeofday

case "$timeofday" in
    "yes" | "y" | "Yes" | "YES" )
            echo "Good Morning"
            echo "Up bright and early this morning?"
            ;;
    "[nN]*" )
            echo "Good Afternoon"
            ;;
    * )
            echo "Sorry, answer not recognised"
            echo "Please answer yes or no"
            exit 1
                ;;
   esac

   exit 0
```

Open Source Technologies

4.4 Cấu trúc lặp

Vòng lặp FOR

```
for <biến> in <giá-trị-1> <giá-trị-2> <giá-trị-3> …
do
        <danh sách lệnh>;
done
```

Ví dụ: Hiển thị các giá trị. Tạo tập tin hienthi có nội dung sau:

```
for gt in apple banana 34
do
        echo $gt;
done
```

Kết quả:

```
apple
banana
34
```

4.4 Cấu trúc lặp

Ví dụ vòng lặp FOR

```
for a in {1..20}
do
    host= 192.168.100.$a
    ping –c2 $host
done
```

```
for i in $(seq 1 2 20)
do
  echo "Welcome $i times"
done
```

Kết quả:

```
Welcome 0 times
Welcome 2 times
Welcome 4 times
Welcome 6 times
Welcome 8 times
Welcome 10 times
```

Open Source Technologies

4.4 Cấu trúc lặp

Vòng lặp WHILE: Thực hiện khi điều kiện đúng, dừng khi điều kiện sai

```
while   <điều kiện>
do
        <danh sách lệnh>;
done
```

Ví dụ: Tạo tập tin vonglapwhile có nội dung sau:

```
chon='y'
while  [ $chon = 'y' ] || [ $chon = 'Y' ]
do
        echo  "Chao ban"
        echo  "An phim Y/y de tiep tuc";
        read  chon
done
```

Open Source Technologies

4.4 Cấu trúc lặp

Ví dụ vòng lặp WHILE

```
i=1
while [[ $i -le 5 ]]
do
   echo “Gia tri $i"
  (( i += 1 ))
done
```

Kết quả:

Gia tri 1
Gia tri 2
Gia tri 3
Gia tri 4
Gia tri 5

```
#!/bin/sh

echo "Enter password"
read trythis

while [ "$trythis" != "secret" ]; do
  echo "Sorry, try again"
  read trythis
done

exit 0
```

Bài tập mở rộng từ ví dụ trên:

1. Nhập pass sai 3 lần => Xuất hiện thông báo lỗi
2. Password không phải là chữ secret mà lấy từ file .txt

Open Source Technologies

4.4 Cấu trúc lặp

Vòng lặp UNTIL: Thực hiện khi điều kiện sai, dừng khi điều kiện đúng

```
until   <điều kiện>
do
        <danh sách lệnh>;
done
```

Ví dụ: Tạo tập tin vonglapuntil có nội dung sau:

```
echo  “Nhap vao so n: ”
read  n
until  [ $n –le  10 ]
do
        echo  “$n lon hon 10”
        (( n -= 1 ))
done
```

5 Các phép toán số học

- let
- Toán tử [ ]
- Toán tử (( ))
- expr
- bc

5 Các phép toán số học

5.1 Toán tử LET

- Toán tử ***let*** được dùng để thực hiện trực tiếp các phép toán cơ bản.
- Khi sử dụng ***let***, chúng ta sử dụng tên biến mà không cần có tiền tố $

Ví dụ:

```
let result=so1 + so2
echo $result
```

Một vài ví dụ:

```
let so+=5
let so-=4
let so—
let so++
```

5 Các phép toán số học

5.2 Toán tử [ ]

- Toán tử [ ] có thể được dùng giống như lệnh let

  result=$[so1 + so2]

- Sử dụng tiền tố $ trong các toán hạng [ ] là hợp lệ

  result=$[$so1 + 7]

5 Các phép toán số học

5.3 Toán tử ( )

- Toán tử (( )) cũng có thể được dùng cho các phép toán số học.
- Tiền tố $ với tên biến được dùng với toán hạng (( ))

Ví dụ:

```
so=15
result=$(( so + 10 ))
```

5 Các phép toán số học

5.3 Biểu thức tính toán EXPR

- Dùng để tính trị các biểu thức số nguyên
- expr được đặt giữa hai dấu ` `
- Trong biểu thức, các toán tử và toán hạng phải cách nhau bằng khoảng trắng
- Cho phép sử dụng các toán tử sau:
  - +, -, *, /, %
  - |, &
  - >, >=. <, <=, !=

Ví dụ:

```
expr 1 + 2
expr 8 / 3
expr 8 % 6
expr 10 \* 2
```

Ví du:

```
result=`expr 3 + 4`
result=$(expr $no1 + 5)
```

Lưu ý: Khi sử dụng EXPR, trước và sau các toán tử phải có khoảng trắng.

5 Các phép toán số học

5.3 Biểu thức tính toán EXPR

- Tất cả các phương pháp trên không hỗ trợ số thực dấu chấm động, và chỉ hoạt động trên các số nguyên.
- ***bc***, 1 cách cho phép tính toán các phép tính sử dụng dấu chấm động và sử dụng các hàm nâng cao.

Ví dụ:

```
echo "4 * 0.56" | bc     => Kết quả 2.24

no=54
result=`echo "$no * 1.5" | bc`
echo $result
=> Kết quả 81.0
```

5.3 Biểu thức tính toán EXPR

Lấy số thập phân với bc

- Trong ví dụ sau, thông số scale = 2 thiết lập số chữ số sau dấu thập phân là 2. Do đó, kết quả xuất ra của bc là 1 số với 2 số thập phân:

Ví dụ: echo "scale=2;3/8" | bc => Kết quả 0.37

- Chúng ta có thể chuyển đổi qua lại giữa các hệ số như nhị phân, bát phân, thập phân … với nhau qua bc:

Ví dụ:

```
no=100
echo "obase=2; $no" | bc
```

=> Kết quả 1100100

```
no=1100100
echo "obase=10; ibase=2; $no" | bc
```

=> Kết quả 100

6 Xử lý chuỗi, văn bản

Lấy chuỗi con trong một chuỗi

- ${string:position}: Lấy chuỗi con từ vị trí position cho đến hết chuỗi.
- ${string:position:n}: trả về n ký tự trong chuỗi string bắt đầu ở vị trí position. Nếu không có tham số n thì lấy từ position đến hết chuỗi.

Ví dụ: string=”this is a substring test”

```
substring=${string:10:9}        # lấy 9 ký tự trong chuỗi string từ vị trí thứ 10
echo $substring                 => kết quả là: substring

var="Welcome to the geekstuff“        # lấy tất cả chuỗi từ vị trí 15
echo ${var:15}                  => kết quả là: geekstuff
```

6 Xử lý chuỗi, văn bản

- Lấy n ký tự đầu chuỗi: ${string:0:n}
- Lấy n ký tự cuối chuỗi: ${string: -n} #Lưu ý: có 1 khoảng trắng trước dấu -
- Lấy chiều dài của chuỗi: ${#string}
- Thay thế chuỗi: *${<string>/<string_to_replace>/<new_string>}*

  Ví dụ: string="Hello, Sir"

  ${string/Sir/Madam}

- Thay thế tất cả các chuỗi tìm kiếm được (replace all)

  ${<string>//<string_to_replace>/<new_string>}

  Ví dụ: string="Xanh Trắng Đen"

  ${string// /-} => *kết quả Xanh-Trắng-Đen*

6 Xử lý chuỗi, văn bản

- Lấy subtring bên phải của string. Nếu lấy chữ cuối thì substring là khoảng trắng.

  ${string##*substring}

  Ví dụ: filename="index.html"

  echo ${filename##*.} => kết quả: html

- Trả về chuỗi bên phải của substring (xóa hết chuỗi bên trái kể cả substring). Nếu xóa chữ đầu tiên thì substring là khoảng trắng.

  ${string#*substring}

  Ví dụ: string="Hi! How are you."

  echo ${string#*How} => kết quả: are you.

  echo ${string#* } => kết quả: How are you.

6 Xử lý chuỗi, văn bản

- Xóa chữ cuối của chuỗi

  substring=${string% *}

- Lấy chữ đầu của chuỗi

  substring=${string%% *}

7 Hàm

Hàm trong shell script

- Shell cho phép chúng ta tạo ra các hàm hoặc thủ tục để thực hiện các công việc.
- Ta cũng có thể gọi các script khác bên trong script đang thực hiện.
- Tuy nhiên, việc gọi script con thường tốn nhiều tài nguyên hơn là gọi hàm.

Cú pháp 1:

```
function  name
{
    statement
}
```

Cú pháp 2:

```
name  ()
{
    statement
}
```

Hàm trong shell script

Ví dụ:

```
func()
{
    echo "Function foo is executing"
}
echo "script starting"
func
echo "script ended"
```

Kết quả:

```
script starting
Function foo is executing
script ended
```

Biến cục bộ và Biến toàn cục

- Biến cục bộ:
  - Chỉ có hiệu lực bên trong hàm
  - Khai báo biến cục bộ ta dùng từ khóa **local** ở phía trước biến.
- Biến toàn cục:
  - Có hiệu lực trên toàn bộ chương trình.
  - Khai báo bình thường, không cần dùng thêm bất kỳ từ khóa nào.

Ví dụ về Biến cục bộ - Biến toàn cục

```
#!/bin/sh

sample_text="global variable"

foo() {
    local sample_text="local variable"

    echo "Function foo is executing"
    echo $sample_text
}

echo "script starting"
echo $sample_text

foo

echo "script ended"
echo $sample_text

exit 0
```

Kết quả:

```
script starting
global variable
Function foo is executing
local variable
script ended
global variable
```

Giá trị trả về của hàm

- Hàm có thể trả về một giá trị, để trả về giá trị số, ta dùng lệnh return.

```
foo()
{
        .........
        return 0
}
```

- Để trả về giá trị chuỗi, ta dùng lệnh echo rồi chuyển hướng nội dung của hàm.

```
foo()
{
        echo “string value”
}
x=$( foo )
```

7 Hàm

Giá trị trả về của hàm

```
#!/bin/bash

tong()
{
        sum=$[ $1 + $2 ]
        return $sum
}

tong 5 8

echo “Tong: $sum”
```

```
#!/bin/bash

tong()
{
        sum=$[ $1 + $2 ]
        echo $sum
}

s=$(tong 5 8)

echo “Tong: $s”
```

Truyền tham số trong hàm

Shell không dùng cách khai báo tham số cho hàm như các ngôn ngữ lập trình khác.

Việc truyền tham số cho hàm, tương tự như truyền tham số trong dòng lệnh.

Ví dụ: ta truyền tham số cho hàm sau

```
echo “User: $1";
echo “Tuoi: $2";
echo “Ten nguoi dung: $3";
```

Thực hiện chạy script có truyền tham số

./user.sh sinhvien 20 “Nguyen Thanh Binh”

Kết quả:

User: sinhvien
Tuoi: 20
Ten nguoi dung: Nguyen Thanh Binh

Như vậy: sinhvien sẽ tương ứng với tham số thứ 1
20 tương ứng tham số thứ 2
Nguyen Thai Binh tương ứng tham số thứ 3

8 Mảng

Khai báo mảng

Cách 1: a=(4 -1 2 66 10)

a=(mot hai ba bon nam sau bay “muoi mot” “muoi hai”)

a=(2 4 nam tam)

Cách 2: array_var[0]="test1“

array_var[1]="test2“

array_var[2]="test3“

array_var[3]="test4”

Đếm số phần tử có trong mảng

echo ${#a[@]} Hoặc echo ${#a[*]}

Truy xuất giá trị các phần tử

Truy xuất tất cả giá trị của các phần tử

echo ${array[@]} Hoặc echo ${array[*]}

Truy xuất giá trị phần tử thứ i của mảng. Bắt đầu bằng vị trí 0

echo ${array[i]}

Xuất ra giá trị các phần tử từ i đến hết mảng

echo ${array[@]:3} *# In các phần tử từ vị trí thứ 3 đến hết mảng*

Xuất ra giá trị của m phần tử từ vị trí n

echo ${array[@]:2:6} *# In phần 6 phần tử từ phần tử thứ 2*

Xuất các giá trị từ cuối mảng trở lên => thêm dấu trừ (-), bắt đầu từ 1

echo ${array[-1]} *# In phần tử cuối mảng*

Các thao tác cơ bản trên mảng

Gán giá trị cho cá phần tử trong mảng

```
array[2]=4          #Gán giá trị 4 cho phần tử thứ 2
```

Thêm vào cuối mảng

```
array1=( ${array[@]}  5 )          # Thêm 5 vào cuối mảng array
array0[${#array0[*]}]=1            # Thêm 1 vào cuối mảng
```

Xóa các phần tử trong mảng (xóa đầu, xóa cuối, xóa bất kỳ)

```
unset array[2]          # Xóa phần tử thứ 2
unset array             # Xóa tất cả các phần tử
```

Sao chép mảng

```
array2="${array1[@]}"
```

Tổng kết

Các phần đã tìm hiểu qua

- Giới thiệu Shell và các loại shell
- Ngôn ngữ lập trình Shell
  - Tham số và Biến số trong Shell: biến thông thường (biến cục bộ, biến toàn cục), biến môi trường.
  - Cấu trúc điều khiển: test, for, while, until
  - Các phép toán số học: let, [], (), expr
  - Xử lý chuỗi, văn bản
  - Hàm
  - Mảng